



UFSM



COLÉGIO
POLITÉCNICO
UFSM

UNIVERSIDADE FEDERAL DE SANTA MARIA

COLÉGIO POLITÉCNICO DA UFSM

CURSO SUPERIOR DE TECNOLOGIA EM SISTEMAS PARA INTERNET

MATHEUS GABRIEL FLECK DE MELLO

GESTÃO DE RISCO — APLICATIVO MOBILE

Apresentação referente à disciplina de Programação para Dispositivos Móveis, do Curso Superior de Tecnologia em Sistemas para Internet, da Universidade Federal de Santa Maria, cursada em regime de autodidatismo.

Santa Maria, RS

2026

1 CONTEXTO E ORIGINALIDADE

O sistema web “Gestão de Risco UFSM” é um projeto autoral: foi desenvolvido durante a disciplina de engenharia de software 1, está registrado institucionalmente e está para entrar em produção na PROPLAN . Este trabalho é a sua extensão para Android — consumindo a mesma API REST, com paridade de papéis de acesso, adicionando o que somente um aplicativo nativo entrega: câmera, GPS e modo offline

Repositório do sistema original:

<https://github.com/matheuskotayko/gestao-riscos-ufsm>

Projeto registrado — Portal de Projetos UFSM (**idProjeto 429407**):

<https://portal.ufsm.br/projetos/publico/projetos/view.html?idProjeto=429407>

2 MOTIVAÇÃO

O problema real acontece longe da mesa: um risco é identificado em campo — num laboratório, em parte isolado do campus, num prédio administrativo — e o gestor precisa registrá-lo no local, sem depender de voltar ao computador.

- a) **Cadastro depende do computador:** hoje o risco só é registrado quando o gestor volta à mesa
- b) **Sem evidência fotográfica:** o sistema web não tem como anexar foto tirada na hora
- c) **Sem localização do risco:** não é possível registrar onde, no campus, o problema foi visto
- d) **Inútil sem conexão:** diversas unidades não têm conexão sem fio estável

3 OBJETIVOS

3.1 OBJETIVO GERAL

Portar a API REST do sistema web para um aplicativo Android nativo(Flutter/Dart), mantendo os três papéis de acesso do sistema original, com funcionamento sem conexão e uso de recursos exclusivos do aparelho.

3.2 OBJETIVOS ESPECÍFICOS

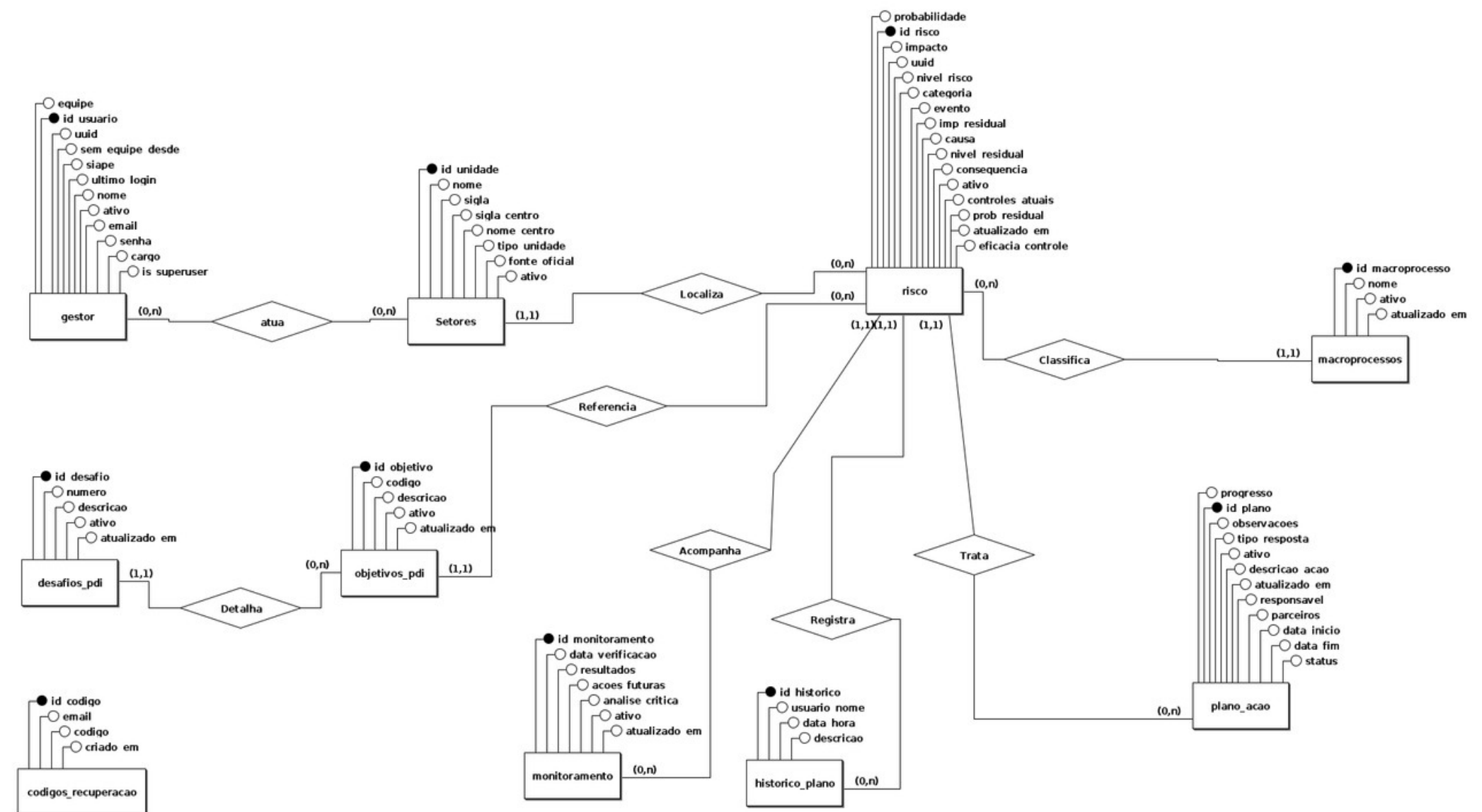
- a) Consumir a API REST via autenticação por token, sem duplicar regras de negócio
- b) Implementar cache local e fila de sincronização com resolução de conflito
- c) Capturar GPS e resolver o endereço por geocodificação reversa
- d) Anexar foto de evidência pela câmera, com envio a um repositório de objetos

4 MODELAGEM DE DADOS

4.1 MODELO CONCEITUAL

- a) recorte do domínio que o aplicativo consome, na notação de Chen (1976);
- b) dez entidades: risco, setor, usuário, plano de ação, monitoramento, histórico, desafio, objetivo, macroprocesso e código de recuperação;
- c) relação N:N entre usuário e setor, que define a permissão de escrita;
- d) o risco concentra os vínculos: setor, objetivo do PDI e macroprocesso.

Figura 1 – Modelo conceitual (MER)



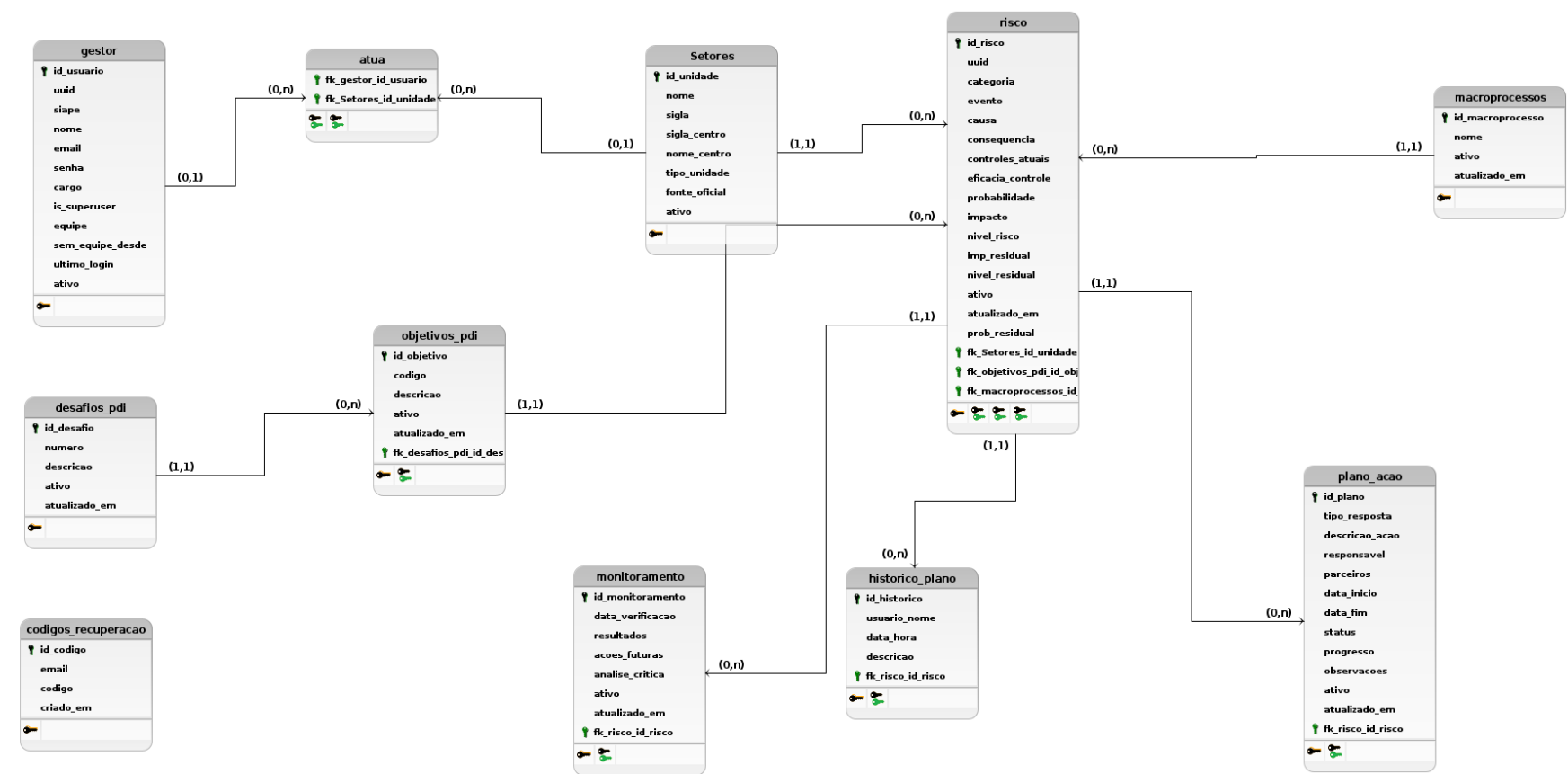
Fonte: Elaborado pelo autor (2026).

4 MODELAGEM DE DADOS

4.2 MODELO LÓGICO

- a) esquema relacional derivado das migrations do Django;
- b) chave primária em todas as tabelas e UUID público nos recursos expostos pela API, preservando o identificador interno;
- c) tabela associativa usuario_setores para a relação N:N;
- d) exclusão lógica por campo ativo, sem remoção física de registro;
- e) no celular, o mesmo recorte é espelhado em tabelas SQLite de cache.

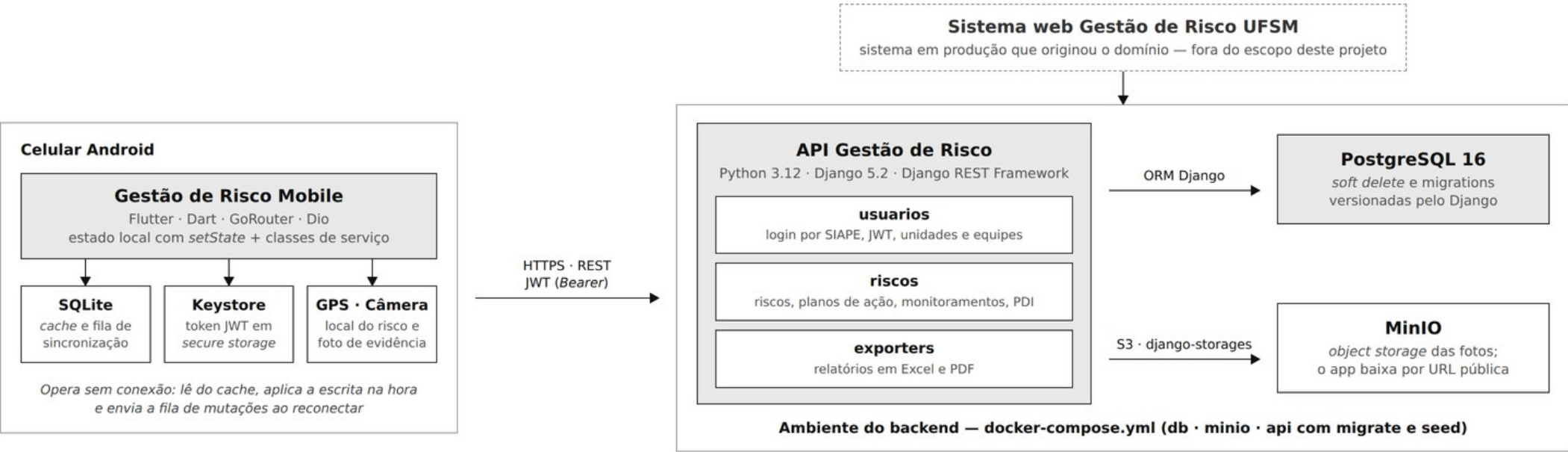
Figura 2 – Modelo lógico (DER)



Fonte: Elaborado pelo autor (2026).

5 ARQUITETURA

Figura 3 – Visão geral do sistema com o aplicativo



Fonte: Elaborado pelo autor (2026).

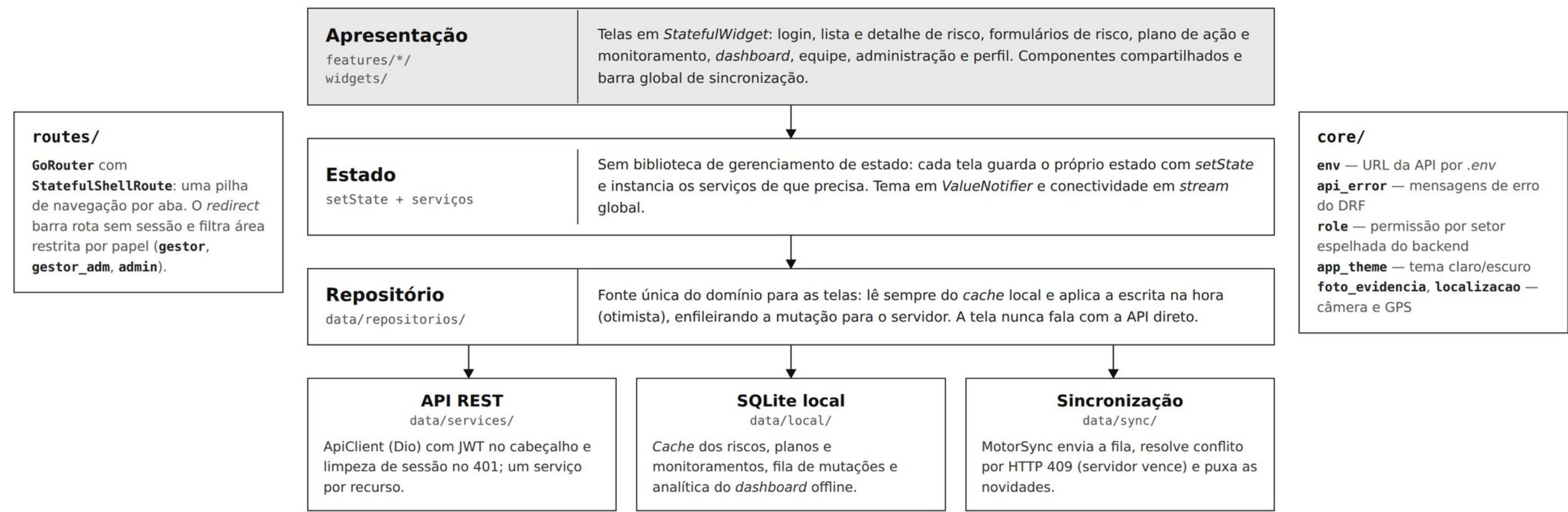
5.1 VISÃO GERAL

- a) o aplicativo conversa apenas com a API, por HTTPS, com o JWT no cabeçalho Bearer;
- b) sistema web e aplicativo consomem a mesma API e as mesmas regras de acesso;
- c) SQLite, Keystore, GPS e câmera ficam no aparelho;
- d) o backend sobe com Docker Compose: banco, API, armazenamento de objetos e massa de demonstração.

5 ARQUITETURA

5.2 CAMADAS DO APLICATIVO

Figura 4 – Camadas do aplicativo e pastas correspondentes



Fonte: Elaborado pelo autor (2026).

5 ARQUITETURA

Figura 5 – Estrutura de pastas de lib/

```
mobile/lib — 66 arquivos Dart · ~9,6 mil linhas

lib/
├─ main.dart           // ponto de entrada: carrega o .env e monta o MaterialApp.router
├─ routes/             // GoRouter: redirect por sessão e papel, uma pilha por aba
├─ core/               // tema, ambiente, erros, RBAC, câmera, GPS e exportação (12)
├─ data/
│   ├─ models/         // modelos de cada recurso + paginação do DRF (10)
│   ├─ services/       // um cliente REST por recurso + token seguro (13)
│   ├─ local/          // SQLite: cache, fila de sync e analítica offline (4)
│   ├─ repositórios/   // fonte única do domínio de riscos para as telas
│   └─ sync/           // MotorSync (fila e conflito 409) e conectividade
├─ features/
│   ├─ auth/           // login e recuperação de senha
│   ├─ riscos/         // lista com filtros, detalhe em 4 abas e formulário
│   ├─ planos_acao/    // formulário do plano de ação (5W2H)
│   ├─ monitoramentos/ // formulário com foto de evidência
│   ├─ dashboard/     // indicadores, matriz de risco 5x5 e gráficos (fl_chart)
│   ├─ equipe/         // membros da unidade (gestor_adm)
│   ├─ admin/          // gestores, unidades e estrutura do PDI
│   ├─ perfil/         // dados do usuário, tema e logout
│   └─ shell/          // navegação inferior conforme o papel
└─ widgets/           // componentes compartilhados e barra de sincronização (8)

test/                  // 82 testes: sync, conflito, cache, RBAC e telas críticas
```

Fonte: Elaborado pelo autor (2026).

5.3 ORGANIZAÇÃO DO CÓDIGO

- a) 66 arquivos Dart, com cerca de 9,6 mil linhas;
- b) core/: infraestrutura comum a todas as telas;
- c) data/: modelos, acesso à API, SQLite, repositório e sincronização;
- d) features/: uma pasta por funcionalidade, com as telas;
- e) test/: 20 arquivos com 82 casos, espelhando lib/.

5 ARQUITETURA

5.4 PACOTES UTILIZADOS

Quadro 1 – Pacotes do aplicativo e sua função

Pacote	Função no aplicativo
dio	Cliente HTTP com interceptador de token
go_router	Navegação e redirecionamento por sessão e papel
sqflite	Cache local e fila de sincronização
flutter_secure_storage	Token JWT no Keystore do Android
connectivity_plus	Detecção de conexão
geolocator e geocoding	GPS e endereço aproximado sem chave de API
image_picker	Foto de evidência pela câmera ou pela galeria
fl_chart	Gráficos do painel de indicadores
share_plus e path_provider	Compartilhamento dos relatórios gerados
flutter_dotenv	URL da API por ambiente

Fonte: Elaborado pelo autor (2026).

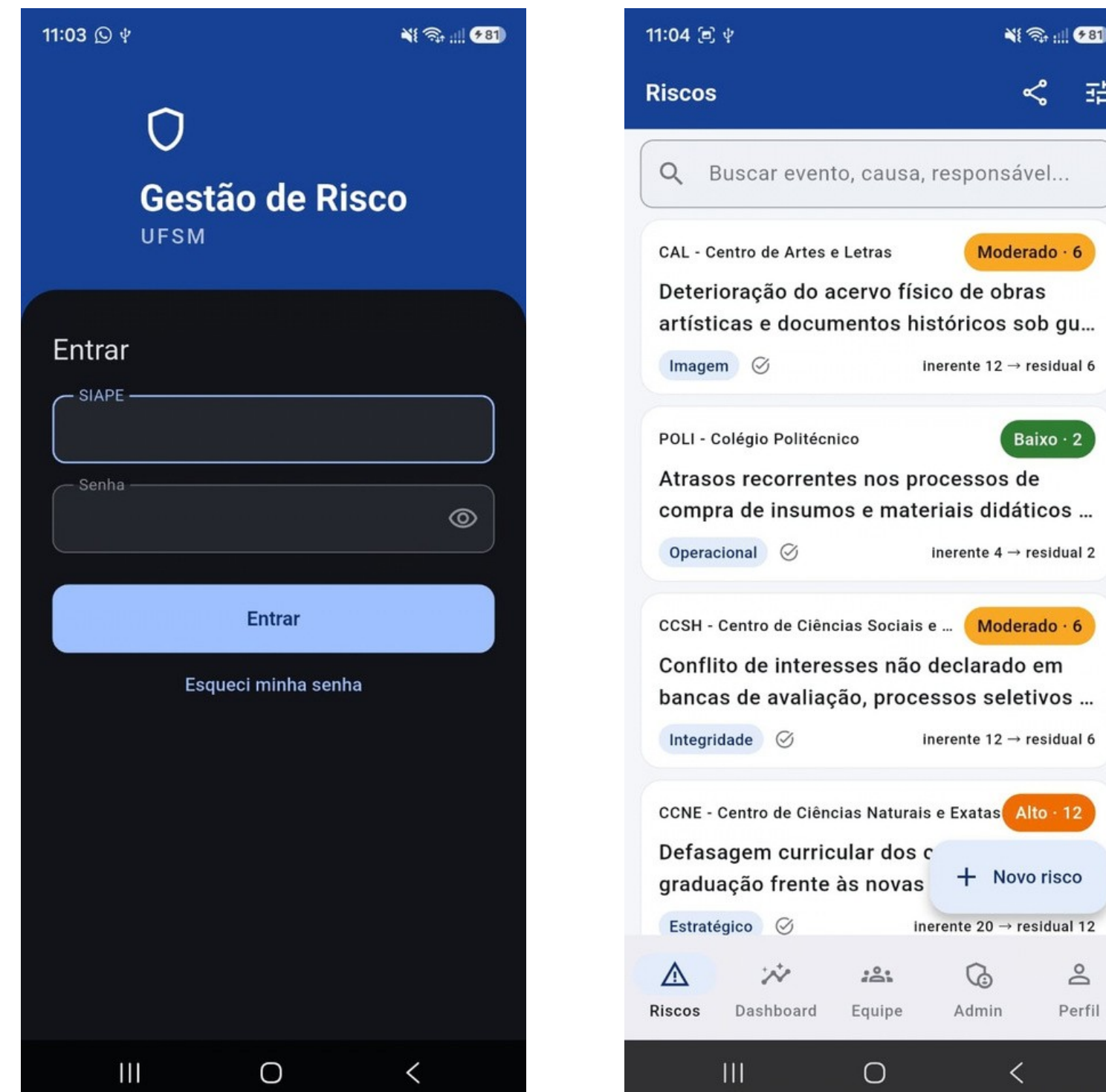
BACKEND CONSUMIDO

Python 3.12, Django 5.2, Django REST Framework, SimpleJWT, PostgreSQL 16, MinIO compatível com S3 e Docker Compose.

6 FUNCIONALIDADES

- a) **Riscos:** cadastro por unidade, categoria e escalas de probabilidade/impacto
- b) **Planos de ação:** estrutura 5W2H, com prazos e progresso
- c) **Monitoramento:** acompanhamento com foto de evidência
- d) **Dashboard:** indicadores e matriz de risco residual 5×5
- e) **Estrutura PDI:** desafios, objetivos e macroprocessos
- f) **Equipe e administração:** gestão de membros, unidades e gestores
- g) **Exportações:** planilhas e relatórios em Excel e PDF
- h) **Segurança:** três papéis de acesso, escrita restrita ao próprio setor

Figura 3 – Tela de Login e Tela Inicial dos Riscos



Fonte: Elaborado pelo autor (2026).

7 IMPLEMENTAÇÃO

Figura 6 – Trecho de código: interceptador do cliente HTTP

```
mobile/lib/data/services/api_client.dart
20  _dio.interceptors.add(
21    InterceptorsWrapper(
22      onRequest: (options, handler) async {
23        final token = await _tokenService.getToken();
24        if (token != null && token.isNotEmpty) {
25          options.headers['Authorization'] = 'Bearer $token';
26        }
27        handler.next(options);
28      },
29      onError: (error, handler) async {
30        if (error.response?.statusCode == 401) {
31          await _tokenService.clear();
32        }
33        handler.next(error);
34      },
35    ),
36  );
```

Fonte: Elaborado pelo autor (2026).

7.1 CLIENTE HTTP E SESSÃO

- a) o cliente recebe a URL base do arquivo .env e tempos-limite de conexão e leitura;
- b) o interceptador anexa o cabeçalho Authorization: Bearer a cada requisição;
- c) o token é lido do armazenamento seguro, nunca de variável em memória;
- d) a resposta 401 limpa a sessão e a navegação seguinte cai no login.

7 IMPLEMENTAÇÃO

Figura 7 – Trecho de código: leitura tolerante do armazenamento seguro

```
mobile/lib/data/services/token_service.dart
24  Future<String?> getToken() => _lerSeguro(_keyToken);
25
26  /// le do secure storage tolerando dado cifrado ilegível (tipo quando a
27  /// chave do android keystore fica dessincronizada depois de reinstalar
28  /// o app). sem isso, uma falha de decriptacao no redirect do go_router
29  /// trava a tela preta pra sempre em vez de so cair no login.
30  Future<String?> _lerSeguro(String chave) async {
31    try {
32      return await _storage.read(key: chave);
33    } catch (_) {
34      await _storage.deleteAll();
35      return null;
36    }
37  }
38
39  Future<void> clear() => _storage.deleteAll();
```

Fonte: Elaborado pelo autor (2026).

7.2 SESSÃO SEGURA NO KEYSTORE

- a) o token e os dados do usuário ficam no Keystore do Android;
- b) o JWT tem vida longa e não há renovação: encerrar a sessão é apagar o armazenamento;
- c) a leitura tolera dado cifrado ilegível, situação que ocorre quando a chave do Keystore é recriada ao reinstalar o aplicativo;
- d) sem esse tratamento, a falha de decriptação travava o roteador numa tela preta.

7 IMPLEMENTAÇÃO

Figura 8 – Trecho de código: proteção das rotas por sessão e papel

```
mobile/lib/routes/app_router.dart
23  redirect: (context, state) async {
24    final logado = await tokenService.hasToken();
25    final indoParaLogin = state.matchedLocation == '/login';
26
27    if (!logado) return indoParaLogin ? null : '/login';
28    if (indoParaLogin) return '/riscos';
29
30    // gating por papel pras areas restritas
31    final role = (await tokenService.getUsuario())?.role ?? Role.gestor;
32    final loc = state.matchedLocation;
33    if (loc.startsWith('/admin') && !role.ehAdmin) return '/riscos';
34    if (loc.startsWith('/equipe') && !role.podeGerenciarEquipe) {
35      return '/riscos';
36    }
37    return null;
38  },
```

Fonte: Elaborado pelo autor (2026).

7.3 NAVEGAÇÃO E PROTEÇÃO DAS ROTAS

- a) o redirecionamento roda antes de cada navegação e consulta a sessão guardada;
- b) sem token, qualquer rota leva ao login; com token, o login leva à lista de riscos;
- c) as áreas restritas são filtradas por papel: administração exige administrador e equipe exige gestor_adm;
- d) nenhuma exceção pode escapar desse trecho, sob pena de travar o roteador.

7 IMPLEMENTAÇÃO

Figura 9 – Trecho de código: permissão de escrita por setor

```
mobile/lib/core/role.dart
20  /// espelha a permissao PertenceAoSetorDoRisco do backend: escrita num risco
21  /// so e permitida se o setor do risco estiver entre os setores do usuario.
22  /// o backend continua sendo a fonte de verdade, isso aqui so controla a ui.
23  ///
24  /// atencao: nem admin (is_superuser) passa direto aqui de proposito - o
25  /// backend nao da bypass pra superusuario nessa regra, entao a ui tambem
26  /// nao deve dar.
27  bool podeEscreverNoSetor(int setorId, List<int> setoresDoUsuario) {
28    return setoresDoUsuario.contains(setorId);
29  }
```

Fonte: Elaborado pelo autor (2026).

7.4 PERMISSÃO ESPELHADA DO BACKEND

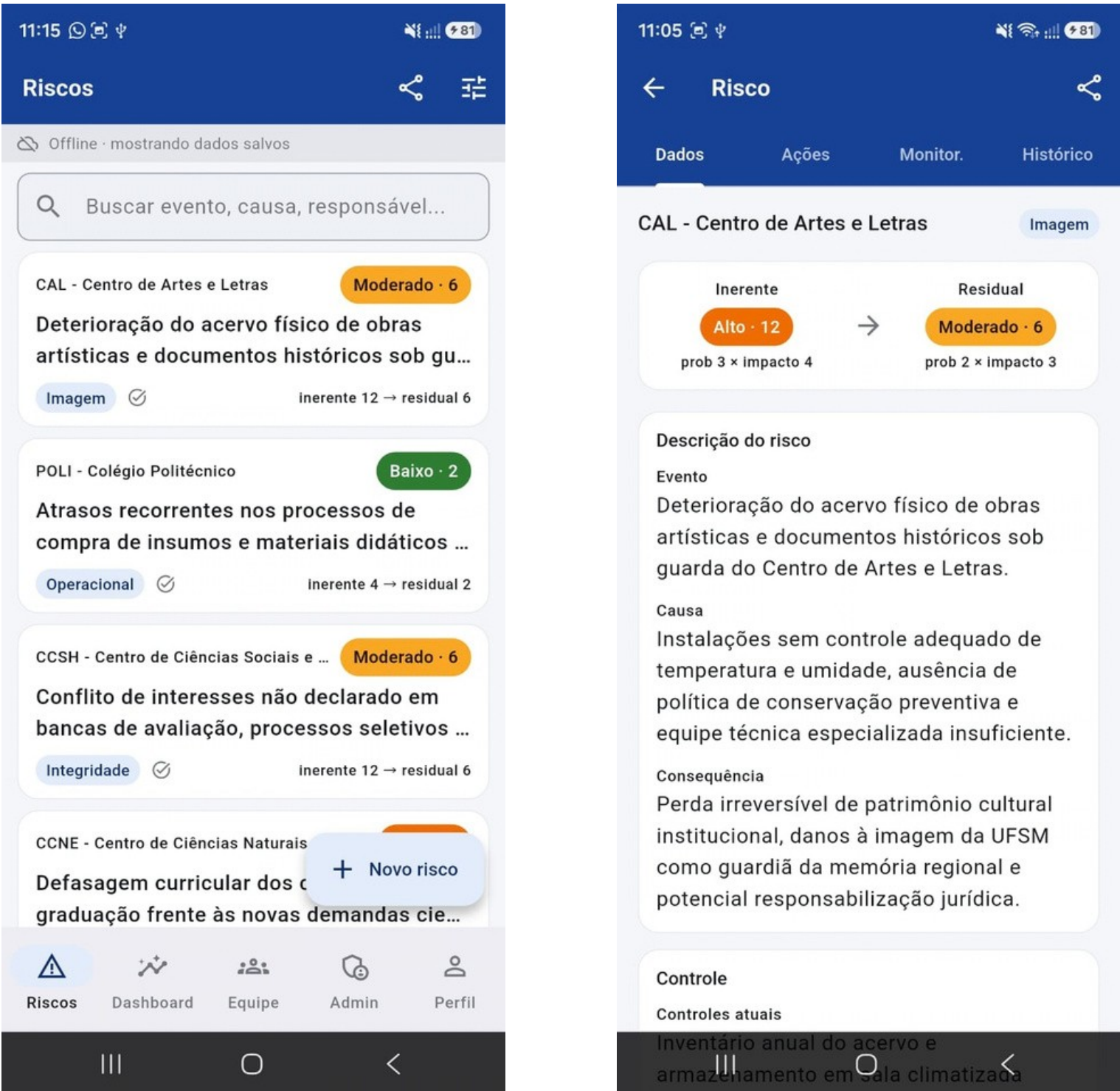
- a) a interface reproduz a permissão de escrita aplicada pelo backend;
- b) a escrita só é liberada quando o setor do risco está entre os setores do usuário;
- c) o administrador não recebe exceção, porque o backend também não concede;
- d) o servidor continua sendo a fonte de verdade: a regra local apenas evita a ação que resultaria em erro 403.

8 MODO OFFLINE

8.1 VISÃO GERAL

- a) **Leitura local:** a lista, o detalhe e o painel são sempre lidos do banco local (sqlite)
- b) **Escrita otimista:** a alteração aparece de imediato e entra numa fila de envio
- c) **Sincronização incremental:** o aplicativo baixa apenas os registros alterados desde a última sincronização
- d) **Concorrência otimista:** uma atualização enviada com versão desatualizada é rejeitada; o servidor prevalece

Figura 4 – Tela de riscos offline

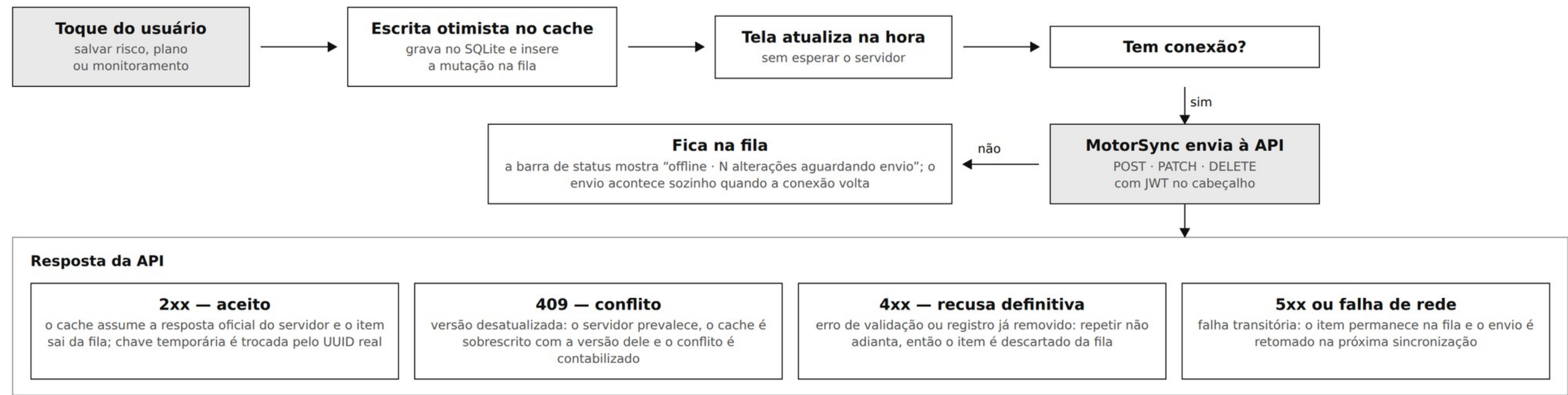


Fonte: Elaborado pelo autor (2026).

8 MODO OFFLINE

8.2 FLUXO DE ALTERAÇÃO E SINCRONIZAÇÃO

Figura 10 – Fluxo de uma alteração, do toque do usuário à sincronização



Fonte: Elaborado pelo autor (2026).

8 MODO OFFLINE

Figura 11 – Trecho de código: leitura a partir do cache local

```
mobile/lib/data/repositorios/risco_repositorio.dart
29 Future<List<Risco>> listar() async {
30   var linhas = await _dao.riscos();
31   if (linhas.isEmpty && Conectividade.instance.online) {
32     // primeira carga: espera o pull terminar antes de mostrar a lista,
33     // senao a tela pisca vazia por um instante
34     await MotorSync.instance.sincronizar();
35     linhas = await _dao.riscos();
36   }
37   // leitura pura do cache – quem quiser atualizar chama o MotorSync
38   return linhas.map(Risco.fromJson).toList();
39 }
```

Fonte: Elaborado pelo autor (2026).

8.3 LEITURA: CACHE E REDE

- a) a lista, o detalhe e o painel são lidos do banco local, sempre;
- b) na primeira carga com conexão, a sincronização é aguardada para a tela não aparecer vazia;
- c) sem conexão, o conteúdo em cache é exibido normalmente;
- d) a atualização é disparada ao abrir a tela, no puxar-para-atualizar e ao reconectar.

8 MODO OFFLINE

Figura 12 – Trecho de código: escrita otimista e fila

```
ata/repositorios/risco_repositorio.dart  
negativo de proposito: id real do servidor sempre e positivo, entao um  
id temporario negativo nunca colide com um registro que ja veio de la  
_tempId() => -DateTime.now().millisecondsSinceEpoch;  
  
ure<void> criarRisco(Map<String, dynamic> payload) async {  
inal chave = 'local-${DateTime.now().microsecondsSinceEpoch}';  
inal json = _comNiveis(payload)  
  ..['uuid'] = chave  
  ..['ativo'] = true;  
wait _dao.salvarLocal(Recurso.risco, chave, json);  
}
```

Fonte: Elaborado pelo autor (2026).

8.4 ESCRITA OTIMISTA

- a) a alteração é gravada no cache e enfileirada antes de qualquer envio;
- b) a tela mostra o novo estado de imediato, mesmo sem rede;
- c) registros criados sem conexão recebem chave temporária, trocada pelo identificador real depois do envio;
- d) o identificador temporário é negativo, o que impede colisão com os do servidor.

8 MODO OFFLINE

Figura 13 – Trecho de código: resolução de conflito na sincronização

```
mobile/lib/data/sync/motor_sync.dart
137     try {
138         final res = await _dio.patch(
139             '$path${item.chave}/',
140             data: await _corpo(body, fotoLocal),
141         );
142         await _forcarDoServidor(item.recurso, res.data);
143     } on DioException catch (e) {
144         if (e.response?.statusCode == 409) {
145             _conflitosSessao++;
146             final atual = e.response!.data['atual'];
147             if (atual != null) {
148                 await _forcarDoServidor(item.recurso, atual);
149             }
150         } else {
151             rethrow;
152         }
153     }
```

Fonte: Elaborado pelo autor (2026).

8.5 CONFLITO E REENVIO

- a) o envio percorre a fila na ordem em que as alterações foram feitas;
- b) a resposta 409 indica versão desatualizada: o servidor prevalece e o cache é sobrescrito com a versão dele;
- c) a recusa definitiva (4xx) descarta o item, porque repetir não mudaria o resultado;
- d) falha de rede ou 5xx mantém o item na fila para a tentativa seguinte.

9 RECURSO NATIVO — GPS

9.1 PERMISSÃO E DADOS REGISTRADOS

Ao selecionar “Usar localização atual”, o aplicativo aciona o GPS do aparelho, solicita a permissão de localização e resolve o endereço aproximado por meio do geocodificador nativo do sistema operacional, sem uso de chave de API externa. O risco passa a registrar coordenadas e um atalho que abre o aplicativo de mapas do sistema.

- a) A captura é opcional e não impede o cadastro do risco
- b) As coordenadas e o endereço seguem o mesmo fluxo de sincronização offline dos demais campos

Figura 5 – Localização via GPS

A imagem é uma captura de tela de um aplicativo móvel, especificamente a tela de criação de um novo risco. No topo, há uma barra azul com o texto "Novo risco" e um ícone de fechar. Abaixo, há uma seção "Localização (opcional)" com o texto "Onde o risco foi identificado no campus." e um endereço "R. Eugênio Mussoi, 927 – Santos" com as coordenadas "-29.71732, -53.81449". Há um botão "Remover" e um botão "Atualizar localização". Abaixo disso, há duas seções de sliders: "Risco inerente" com "Probabilidade" e "Impacto" ambos ajustados para "3 · Médio", resultando em uma "Faixa: Moderado"; e "Risco residual" com "Probabilidade" e "Impacto" ambos ajustados para "2 · Baixo", resultando em uma "Faixa: Moderado · 4". No fundo, há um botão azul "Criar risco".

Fonte: Elaborado pelo autor (2026).

9 RECURSO NATIVO — GPS

Figura 14 – Trecho de código: captura da localização com permissão

```
mobile/lib/core/localizacao.dart
29  if (!await habilitado()) {
30    throw 'Ative o GPS do celular para registrar a localização.';
31  }
32
33  var permissao = await checar();
34  if (permissao == LocationPermission.denied) {
35    permissao = await pedir();
36  }
37  if (permissao == LocationPermission.denied ||
38      permissao == LocationPermission.deniedForever) {
39    throw 'Permissão de localização negada.';
40  }
41
42  final p = await posicao();
43  return (latitude: p.latitude, longitude: p.longitude);
```

Fonte: Elaborado pelo autor (2026).

9.2 CAPTURA DA LOCALIZAÇÃO

- a) confere se o serviço de localização do aparelho está ligado;
- b) solicita a permissão em tempo de execução, apenas no momento do uso;
- c) trata a permissão negada e a negada permanentemente com mensagem própria;
- d) lê a posição com alta precisão e resolve o endereço pelo geocodificador nativo, sem chave de API externa.

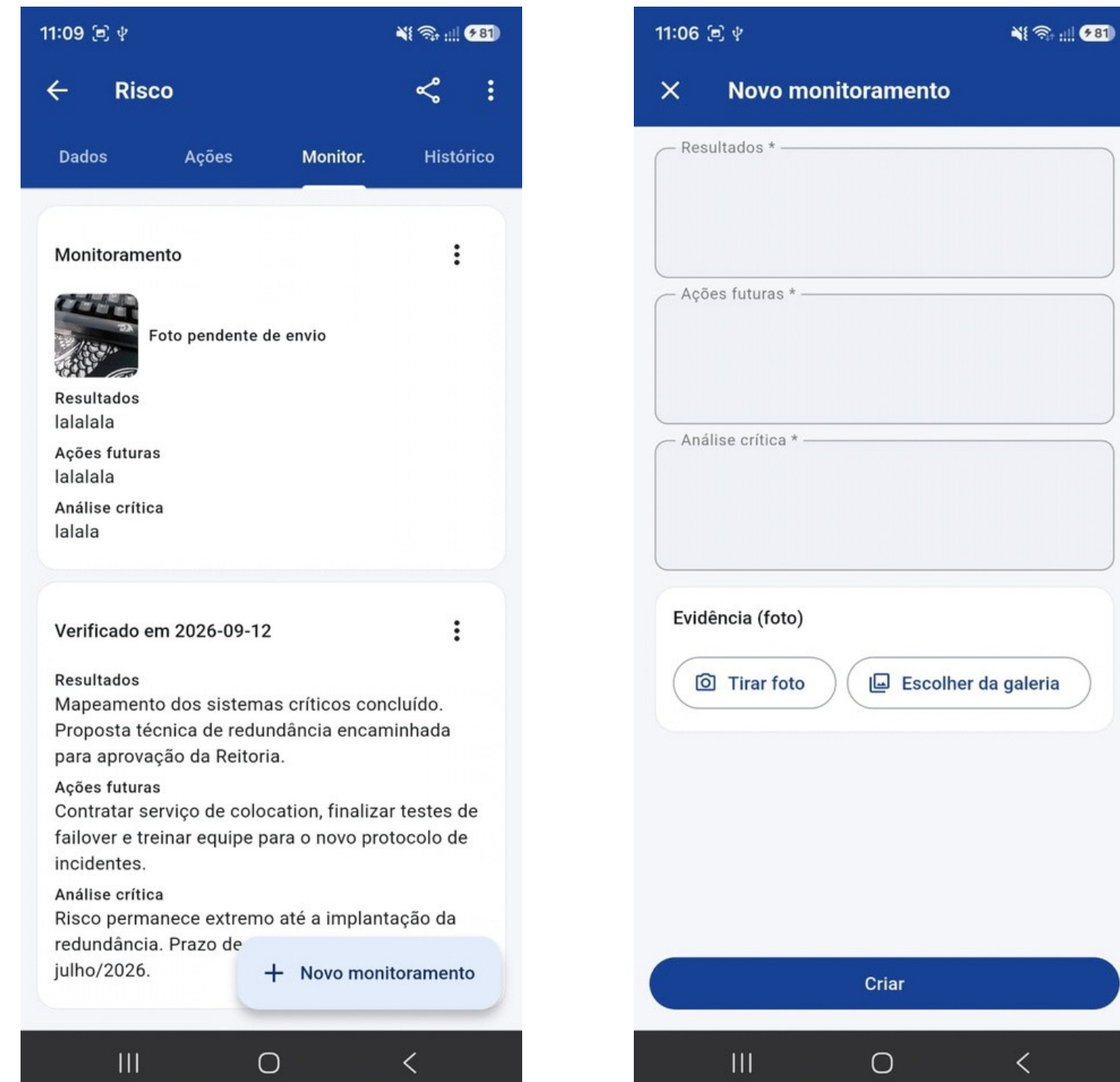
10 RECURSO NATIVO — CÂMERA

10.1 FOTO DE EVIDÊNCIA

No momento da verificação, o gestor registra a situação real do local. A imagem pode vir da câmera, tirada na hora, ou da galeria do aparelho, quando a foto já havia sido feita durante a inspeção. O arquivo é enviado por multipart/form-data a um repositório de objetos compatível com S3 (MinIO).

- a) a foto é opcional, no máximo uma por monitoramento;
- b) funciona sem conexão: o arquivo fica no aparelho e sobe junto com o monitoramento quando a rede volta;
- c) a imagem é reduzida no próprio aparelho antes do envio.

Figura 6 – Captura de foto de evidência



Fonte: Elaborado pelo autor (2026).

10 RECURSO NATIVO — CÂMERA

Figura 15 – Trecho de código: captura da foto de evidência

```
mobile/lib/core/foto_evidencia.dart
11 Future<String?> tirarFotoEvidencia({
12   ImageSource origem = ImageSource.camera,
13   Future<XFile?> Function(ImageSource)? camera,
14 }) async {
15   final tirar = camera ??
16     (ImageSource s) => ImagePicker().pickImage(
17       source: s, maxWidth: 1600, imageQuality: 70,
18     );
19   final foto = await tirar(origem);
20   if (foto == null) return null;
21
22   // copia pra um arquivo permanente: sobrevive se fechar o app
23   // antes de sincronizar
24   final dir = await getApplicationDocumentsDirectory();
25   final destino = p.join(dir.path, 'evidencias', '$agora$ext');
26   await File(foto.path).copy(destino);
27   return destino;
28 }
```

Fonte: Elaborado pelo autor (2026).

10.2 CAPTURA E ENVIO DA FOTO

- a) a origem da imagem é escolhida pelo usuário: câmera ou galeria;
- b) a foto é reduzida no próprio aparelho antes do envio;
- c) o arquivo é copiado para o diretório do aplicativo e sobrevive ao fechamento antes da sincronização;
- d) o envio ocorre por multipart/form-data junto com o monitoramento.

11 SEGURANÇA

11.1 PERFIS E SESSÃO

Quadro 2 – Perfis de acesso e permissões

Papel	Permissões
admin	Acesso irrestrito: cadastro de gestores, gestão de unidades, estrutura do PDI, registros inativos
gestor_adm	Tudo o que o gestor faz, além de gerenciar os membros das equipes das próprias unidades
gestor	CRUD de riscos apenas nas unidades às quais está vinculado; leitura ampla

Fonte: Elaborado pelo autor (2026).

- a) Sessão: token guardado de forma segura no dispositivo (Keystore do Android)
- b) Sem token: requisição recusada (401); autenticado sem o papel exigido (403)
- c) Recuperação de senha: código de 6 dígitos, válido por 1 minuto, de uso único
- d) Sem oráculo: código errado, expirado ou já usado retornam a mesma mensagem genérica

12 QUALIDADE

12.1 TESTES AUTOMATIZADOS

214 testes no backend (pytest, 87% de cobertura de linha) e 82 testes no aplicativo mobile (flutter test), cobrindo a matriz de autorização, invariantes de domínio, casos de borda, regressão dos problemas de sincronização já corrigidos, captura de GPS e envio de foto por multipart.

Quadro 3 – Testes automatizados do aplicativo por área

Área	Testes
Domínio do risco e cálculo de níveis	12
Mensagens de erro da API	8
Cache e fila de sincronização (SQLite)	7
Motor de sincronização e conflito	8
Papéis e permissão por setor	4
GPS e geocodificação reversa	4
Painel e analítica offline	6
Paginação e exportação	4
Telas e componentes compartilhados	29
Total	82

Fonte: Elaborado pelo autor (2026).

12 QUALIDADE

12.2 CRITÉRIOS DE AVALIAÇÃO

- a) **Motivação clara de mobile:** operação em campo, no local em que o risco é identificado, sem depender de um computador;
- b) **Armazenamento offline:** cache local em SQLite, com fila de sincronização e resolução de conflito;
- c) **Envio a servidor web:** API REST em Django, com envio e reconciliação automáticos;
- d) **Autenticação com API REST:** JWT emitido pelo backend, associado ao SIAPE do usuário e guardado no Keystore;
- e) **Recursos nativos do aparelho:** GPS com geocodificação reversa e câmera — dois recursos, não apenas um;
- f) **Qualidade verificável:** 296 testes automatizados no total, sendo 82 no aplicativo.

13 DEMONSTRAÇÃO

Roteiro da demonstração, com a massa de dados criada pelo seed do backend:

- a) autenticar com SIAPE e senha e receber o token;
- b) cadastrar um risco e capturar a localização por GPS;
- c) registrar um monitoramento com foto de evidência;
- d) ativar o modo avião e seguir consultando pelo cache;
- e) alterar um risco sem conexão e ver a pendência na barra de status;
- f) restabelecer a conexão e acompanhar a sincronização automática;
- g) exportar a lista filtrada em Excel ou PDF e compartilhar.

Figura 7 – Captura do vídeo demonstrativo



Fonte: Elaborado pelo autor (2026).

14 CONSIDERAÇÕES FINAIS

O aplicativo leva a gestão de risco para o campo reaproveitando a API do sistema web, sem duplicar regra de negócio, opera sem conexão com sincronização confiável e emprega dois recursos nativos do aparelho — GPS e câmera — aplicados onde fazem sentido no domínio do problema, e não como itens isolados de um requisito.

O que pode ser implementado futuramente

- a) Notificações push para prazos de plano de ação
- b) Sincronização em segundo plano, sem depender do aplicativo aberto
- c) Mapa de calor dos riscos georreferenciados no campus

Sistema web original:

<https://github.com/matheuskotayko/gestao-riscos-ufsm>

Aplicativo mobile:

<https://github.com/matheuskotayko/gestao-riscos-mobile>

Registro institucional:

<https://portal.ufsm.br/projetos/publico/projetos/view.html?idProjeto=429407>

REFERÊNCIAS

- ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. ABNT NBR ISO 31000: gestão de riscos — diretrizes. Rio de Janeiro: ABNT, 2018.
- CHEN, P. P. The entity-relationship model: toward a unified view of data. ACM Transactions on Database Systems, New York, v. 1, n. 1, p. 9-36, mar. 1976.
- GOOGLE. Offline-first support. Flutter documentation. [S. l.], 2026a. Disponível em: <https://docs.flutter.dev/app-architecture/design-patterns/offline-first>. Acesso em: 21 set. 2026.
- GOOGLE. Request location permissions. Android Developers. [S. l.], 2026b. Disponível em: <https://developer.android.com/develop/sensors-and-location/location/permissions>. Acesso em: 21 set. 2026.
- UNIVERSIDADE FEDERAL DE SANTA MARIA. Plano de Desenvolvimento Institucional 2016-2026. Santa Maria: UFSM, 2016. Disponível em: <https://www.ufsm.br/pdi>. Acesso em: 21 set. 2026.